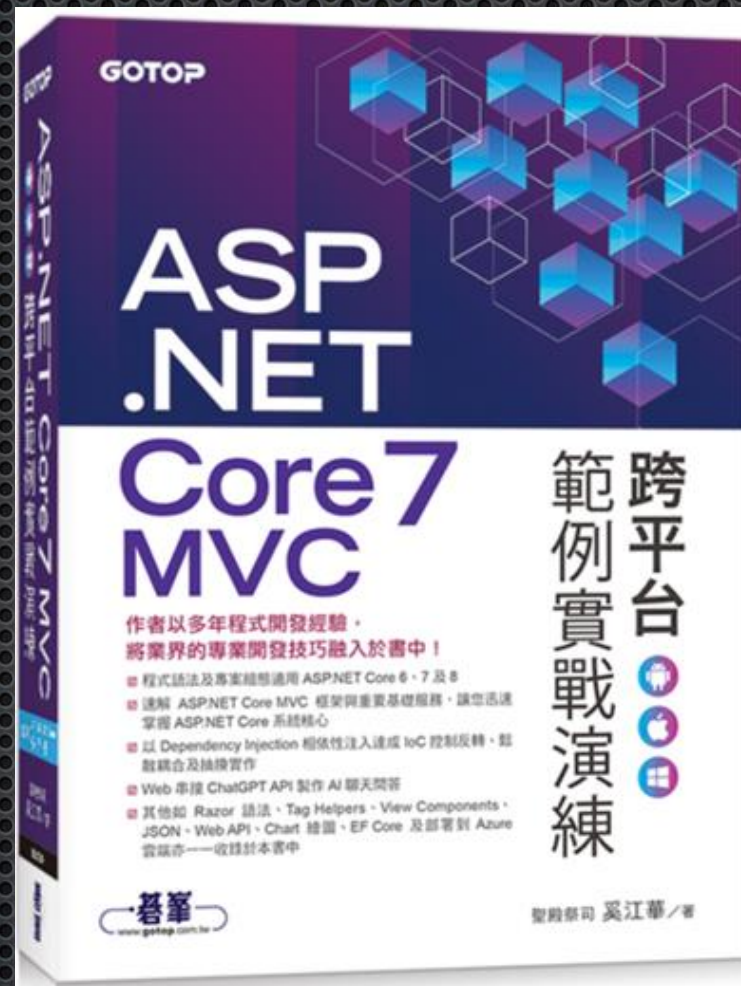


第02章

ASP.NET Core MVC 概觀 與VS 2022開發環境



作者：聖殿祭司。奚江華

大綱(1)

2-1 MVC 樣式 vs. ASP.NET Core MVC 框架

2-2 Visual Studio 2022 開發工具下載及安裝

2-3 ASP.NET Core MVC 框架組成及運作流程

2-4 建立第一個 MVC 專案與檢視六大步驟的對應檔

2-5 掌握 Controller、Model 及 View 的建立技巧

2-6 解析 ASP.NET MVC 專案資料夾功用

大綱(2)

2-7 身分驗證的四種模式

2-8 用 LibMan 管理前端函式庫

2-9 IIS Express 及 SQL Server Express LocalDB 開發環境

2-10 IIS Server 及 Hosting Bundle 安裝

2-11 部署 ASP.NET Core 應用程式至 IIS 網站

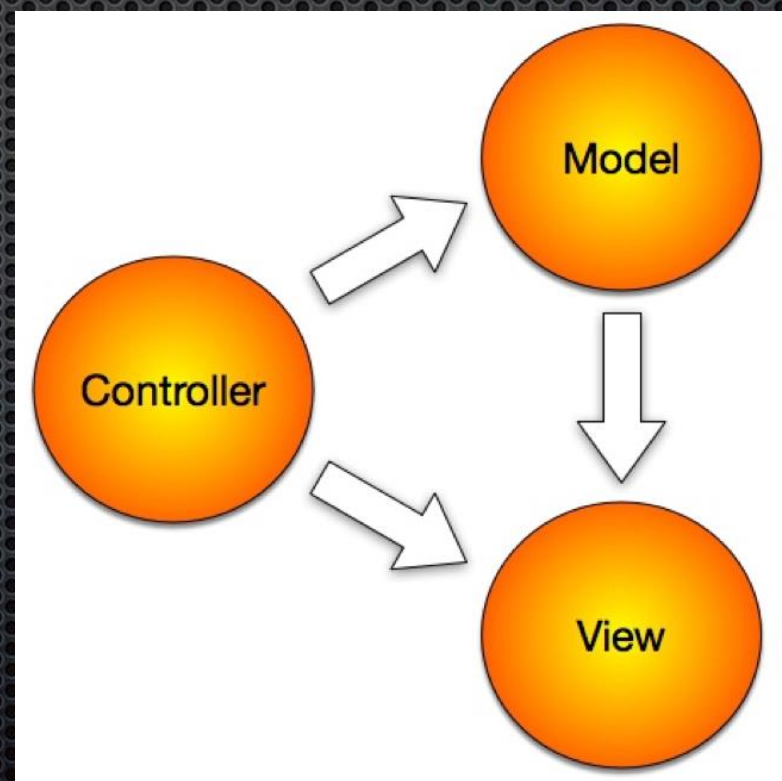
2-12 建立 Model 時常用的 C#物件和集合初始設定式

MVC 樣式 vs. ASP.NET Core MVC 框架

2-1

MVC Pattern

- MVC是一種設計樣式（Design Pattern）
- 代表Model、View和Controller三個部分，Model負責商業邏輯及資料面，View負責UI介面，Controller負責接收Request請求、指揮協調Model和View、回應結果



ASP.NET MVC

- 微軟實踐MVC理論而推出的Web框架
- 將概念上的MVC樣式轉變成具體可行之框架，針對Model、View和Controller提供實作功能，以及週邊配套與輔助機制
- 總結：MVC是設計樣式，而ASP.NET MVC是支持MVC設計的框架

MVC優點

- 達到較好SoC關注點分離 (Separation of Concerns)
- 較好的分層架構
- 降低系統複雜度與提高理解性
- 系統自然比較好維護與擴展

SoC關注點分離

- 而所謂的 SoC 關注點分離是一種設計原則，將一個應用程式分為不同的部分或區塊
- 每個部分都有一個關注點每個關注點內部程式或功能只包含其關心的部分，具體化實現 SoC 的程式稱為模組化系統
- 另一種實現 SoC 的手法則是系統分層設計 (Layered design)，將系統分為展現層、商業邏輯層、資料存取層、資料持久層

Visual Studio 2022

開發工具下載及安裝

2-2

Visual Studio三大產品線

- Visual Studio 2019 (Windows)
 - Community (免費版) 、 Professional 、 Enterprise 及 Test Professional
- Visual Studio Code (Windows 、 Linux 及 Mac)
- VisualStudio for Mac
 - Community (免費版) 、 Professional 、 Enterprise

可安裝VS 2022的作業系統

- Windows 11 版本 21H2 或更高版本
- Windows 10 1909 版或更高版本
- Windows 伺服器 2022：Standard 和 Datacenter
- Windows 伺服器 2019：Standard 和 Datacenter
- Windows Server 2016：Standard 和 Datacenter

VS 2022軟硬體安裝需求：<https://bit.ly/3cezTkF>

可安裝VS 2022的作業系統

- Windows 11 版本21H2 或更高版本：家用、Pro、Pro 教育版、適用於工作站、Enterprise 和教育的 Pro
- Windows 10 1909 版或更高版本：家用版、Professional 版、教育版和 Enterprise
- Windows伺服器2022：Standard和Datacenter
- Windows伺服器2019：Standard和Datacenter
- Windows Server 2016：Standard和Datacenter

<https://visualstudio.microsoft.com/zh-hant/downloads/>

VS 2022 Installer 工作負載

修改中 — Visual Studio Enterprise 2022 — 17.7.3

工作負載 個別元件 語言套件 安裝位置

Web 與雲端 (4)



ASP.NET 與網頁程式開發

使用 ASP.NET Core、ASP.NET、HTML/JavaScript 及容器 (包括 Docker 支援) 建立 Web 應用程式。

1



Python 開發

對 Python 進行編輯、偵錯、互動式開發及原始檔控制。



Azure 開發

用於使用 .NET 和 .NET Framework 開發雲端應用程式及建立資源的 Azure SDK、工具及專案。同時包含用於將應用...



Node.js 開發

使用非同步的事件驅動 JavaScript 執行階段 Node.js 建置可調整的網路應用程式。



傳統型與行動裝置 (5)



NET Multi-Platform App UI 開發

使用 C# 與 .NET MAUI，從單一程式碼基底建置 Android、iOS、Windows 和 Mac 應用程式。



.NET 桌面開發

使用 C#、Visual Basic 及 F#，利用 .NET 和 .NET Framework 建置 WPF、Windows Forms 與主控台應用程式。



使用 C++ 的桌面開發

使用您選擇的工具 (包括 MSVC、Clang、CMake 或 MSBuild) 建置適用於 Windows 的新式 C++ 應用程式。



通用 Windows 平台開發

使用 C#、VB 或選用 C++，來建立適用於通用 Windows 平台的應用程式。



安裝詳細資料

- Visual Studio 核心編輯器
- ASP.NET 與網頁程式開發
- Azure 開發
- ▾ Visual Studio 擴充功能開發
 - ▾ 已包含
 - ✓ Visual Studio SDK
 - ✓ Visual Studio 擴充功能開發必要條件
 - ▾ 選擇性
 - ✓ .NET 分析工具
 - ✓ IntelliCode
 - ✓ IntelliTrace
 - ✓ 文字範本轉換
 - ✓ .NET Compiler Platform SDK
 - ✓ Developer Analytics Tools
 - ☐ Modeling SDK

位置

C:\Program Files\Microsoft Visual Studio\2022\Enterprise

移除不受支援的元件(R)

總共所需空間 19 MB

繼續進行即表示您同意所選 Visual Studio 版本的[授權](#)。我們也可讓您使用 Visual Studio 下載其他軟體。此軟體為分開授權，如同[協力廠商聲明](#)或其隨附的授權中所述。繼續進行即表示您也同意該授權。

2

在下載時安裝

關閉(C)

個別元件

修改中 — Visual Studio Enterprise 2022 — 17.7.3

工作負載 個別元件 語言套件 安裝位置

搜尋元件 (Ctrl+Q)

.NET

- ☐ .NET 5.0 執行階段 (不支援)
- ☐ .NET 6.0 WebAssembly Build Tools
- ☒ .NET 6.0 執行階段 (長期支援)
- ☒ .NET 7.0 執行階段
- ☐ .NET Core 2.1 執行階段 (不支援)
- ☐ .NET Core 3.1 執行階段 (不支援)
- ☐ .NET Framework 3.5 開發工具
- ☐ .NET Framework 4.5.2 目標套件
- ☒ .NET Framework 4.6 目標套件
- ☐ .NET Framework 4.6.1 SDK
- ☐ .NET Framework 4.6.1 目標套件
- ☐ .NET Framework 4.6.2 SDK
- ☐ .NET Framework 4.6.2 目標套件
- ☐ .NET Framework 4.7 SDK
- ☐ .NET Framework 4.7 目標套件
- ☐ .NET Framework 4.7.1 SDK
- ☐ .NET Framework 4.7.1 目標套件

安裝詳細資料

- Visual Studio 核心編輯器
- ASP.NET 與網頁程式開發
- Azure 開發
- ▼ Visual Studio 擴充功能開發
 - ▼ 已包含
 - ✓ Visual Studio SDK
 - ✓ Visual Studio 擴充功能開發必要條件
 - ▼ 選擇性
 - ☒ .NET 分析工具
 - ☒ IntelliCode
 - ☒ IntelliTrace
 - ☒ 文字範本轉換
 - ☒ .NET Compiler Platform SDK
 - ☒ Developer Analytics Tools
 - ☐ Modeling SDK

位置

C:\Program Files\Microsoft Visual Studio\2022\Enterprise

移除不受支援的元件(R)

總共所需空間 19 MB

繼續進行即表示您同意所選 Visual Studio 版本的[授權](#)。我們也可讓您使用 Visual Studio 下載其他軟體。此軟體為分開授權，如同[協力廠商聲明](#)或其隨附的授權中所述。繼續進行即表示您也同意該授權。

在下載時安裝

關閉(C)

ASP.NET Core MVC 框架組成及運作流程

2-3

Model-View-Controller職責

- Controller控制器

負責與使用者的互動，包括接收Request請求，協調Model及View，最終輸出回應給使用者

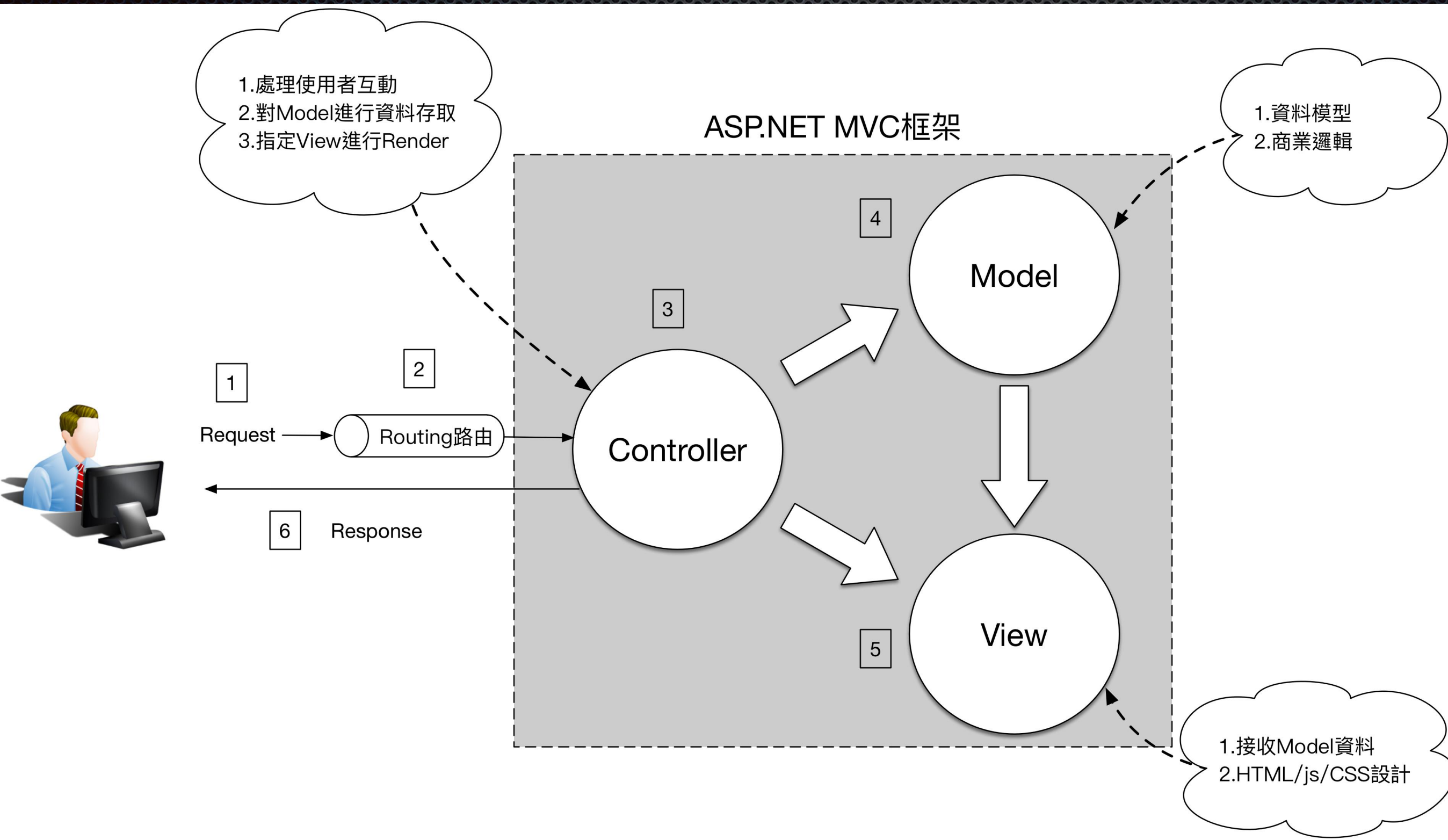
- Model模型

包含資料模型及商業邏輯兩部分，對資料庫的資料存取就是Model職責

- View檢視

負責HTML網頁、JavaScript、CSS、網站佈局等UI介面

ASP.NET Core MVC 框架運作流程



ASP.NET MVC執行六大步驟

- ① 使用者在瀏覽器輸入URL網址後，會發出Request請求至伺服器
- ② 中間經過Routing路由機制，找到對應的Controller及Action Method
- ③ Action呼叫Model，以讀取或更新資料
- ④ Model進行實際的商業邏輯計算與資料庫存取，然後回傳資料給Action
- ⑤ Action將Model資料傳給View作HTML的呈現
- ⑥ View Engine將最終HTML結果寫入Response輸出資料流，回應給使用者瀏覽器

建立第一個 MVC 專案 與檢視六大步驟的對應檔

2-4

建立ASP.NET Core Web應用程式

Visual Studio 2022

開啟最近的项目(R)

開始使用

-  **複製存放庫(C)**
從像是 GitHub 或 Azure DevOps 等這類線上存放庫取得程式碼
-  **開啟專案或解決方案(P)**
開啟本機 Visual Studio 專案或 .sln 檔案
-  **開啟本機資料夾(F)**
瀏覽和編輯任何資料夾內的程式碼
-  **建立新的專案(N)**
透過程式碼 Scaffolding 選擇專案 **1** 開始使用
[不使用程式碼繼續\(W\) →](#)

搜尋範本 (Alt+S)(S)  [全部清除\(C\)](#)

C# 所有平台(P) Web

 **ASP.NET Core Web 應用程式 (Model-View-Controller)**
用於建立 ASP.NET Core 應用程式的專案範本，附有 ASP.NET Core MVC 的檢視及控制器範例。此範本也可用於 RESTful HTTP 服務。
C# Linux macOS Windows 雲端 服務 Web **2**

 **Blazor Server 應用程式**
用來建立 Blazor Server 應用程式的專案範本，該應用程式會在 ASP.NET Core 應用程式內執行伺服器端，並透過 SignalR 連線處理使用者互動。Web 應用程式具有豐富且動態的使用者介面 (UI) 時，此範本最適用。
C# Linux macOS Windows 雲端 Web

 **ASP.NET Core gRPC 服務**
用來建立 gRPC ASP.NET Core 服務的專案範本。
C# Linux macOS Windows 雲端 服務 Web

[上一步\(B\)](#) [下一步\(N\) **3**](#)

建立專案名稱與位置

設定新的專案

ASP.NET Core Web 應用程式 (Model-View-Controller)

C#

Linux

macOS

Windows

雲端

服務

Web

專案名稱(J)

Mvc7_FirstMVC

4

位置(L)

C:\CoreMvc7Examples\

5

...

解決方案名稱(M) ⓘ

Mvc7_FirstMVC

☐ 將解決方案與專案置於相同目錄中(D)

上一步(B)

下一步(N)

6

指定.NET 7架構

其他資訊

ASP.NET Core Web 應用程式 (Model-View-Controller)

C#

Linux

macOS

Windows

雲端

服務

Web

架構(F) ⓘ

.NET 7.0 (標準字詞支援)

7

驗證類型(A) ⓘ

無

☒ 針對 HTTPS 進行設定(H) ⓘ

☐ 啟用 Docker(E) ⓘ

Docker OS ⓘ

Linux

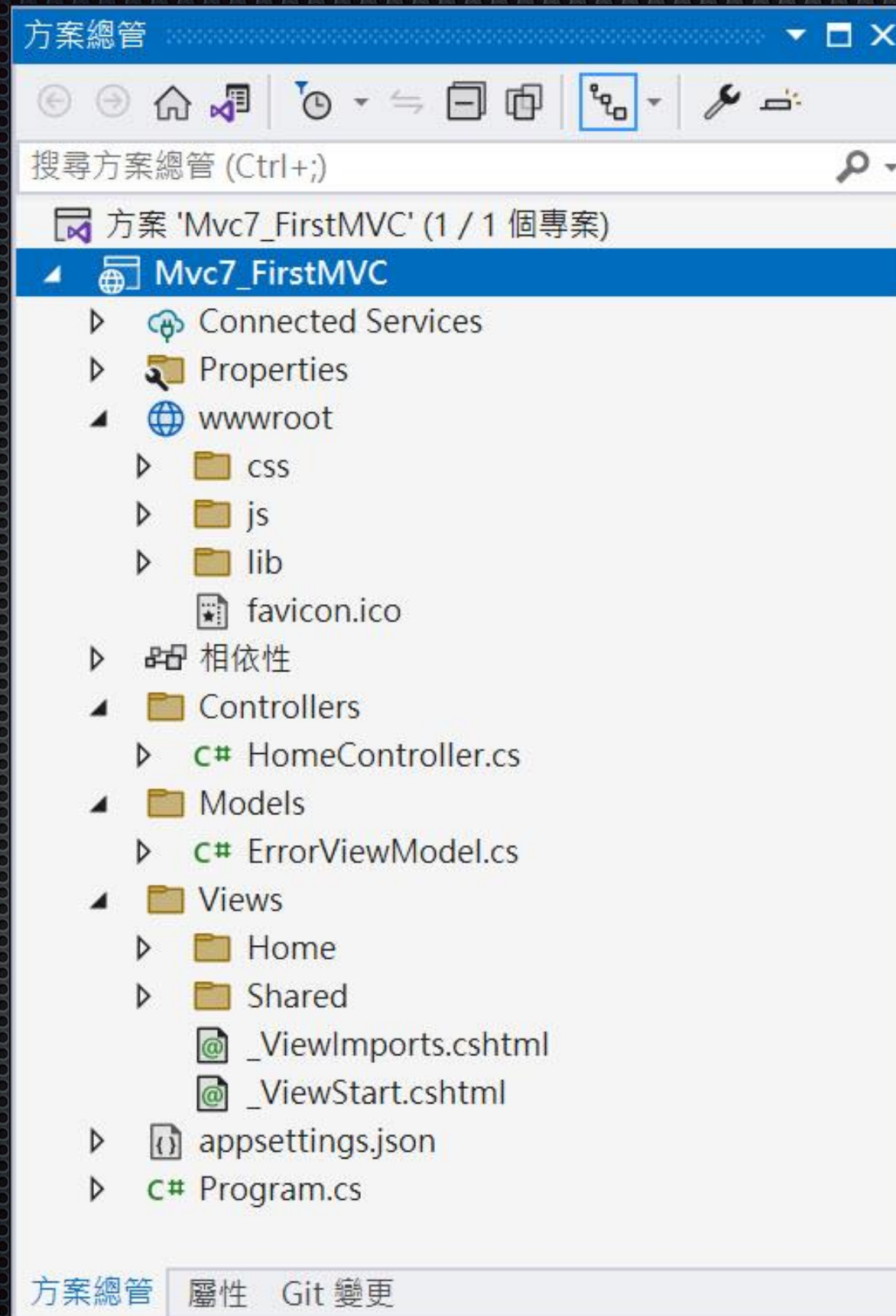
☐ 不要使用最上層陳述式(T) ⓘ

上一步(B)

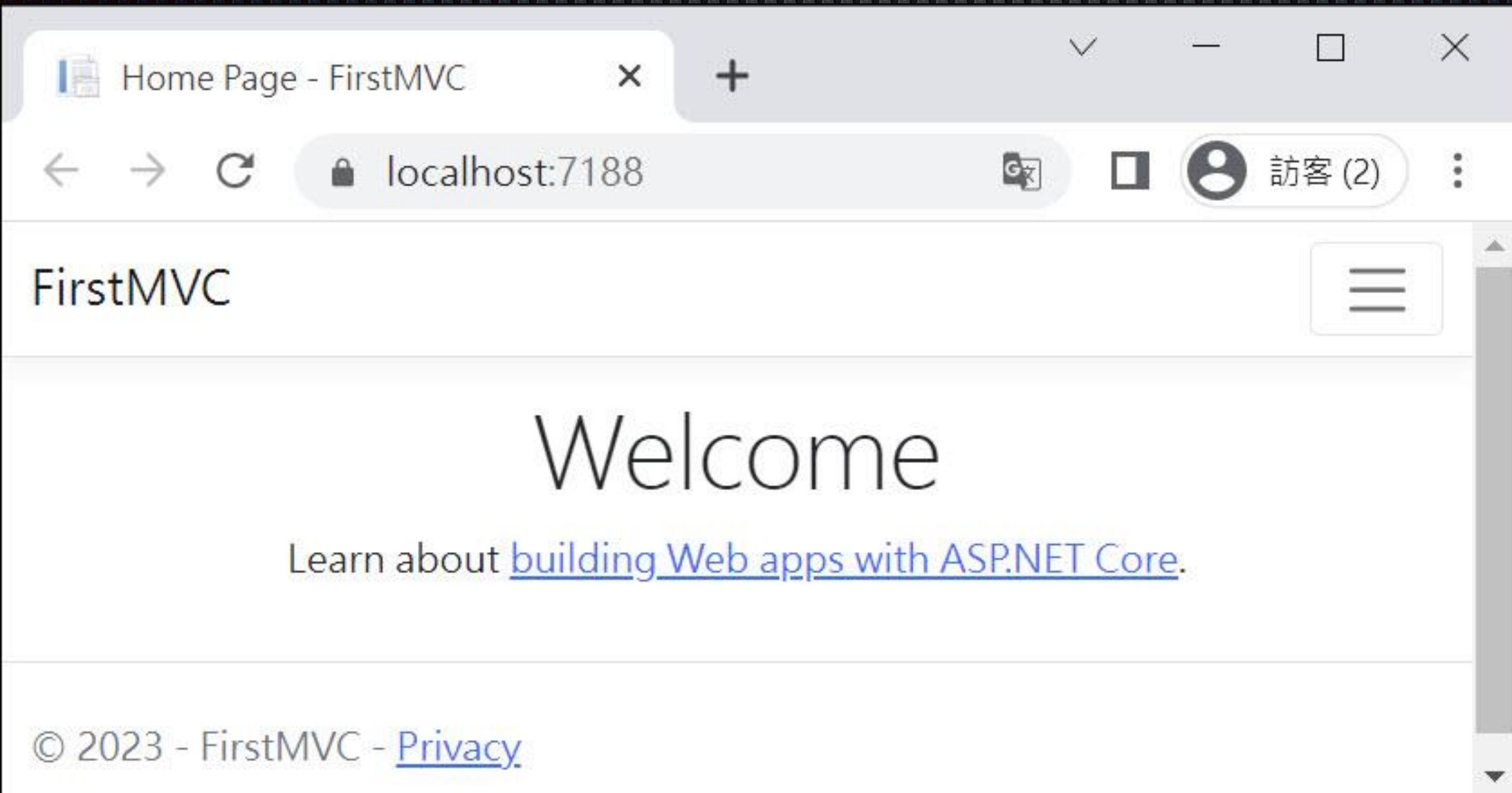
建立(C)

8

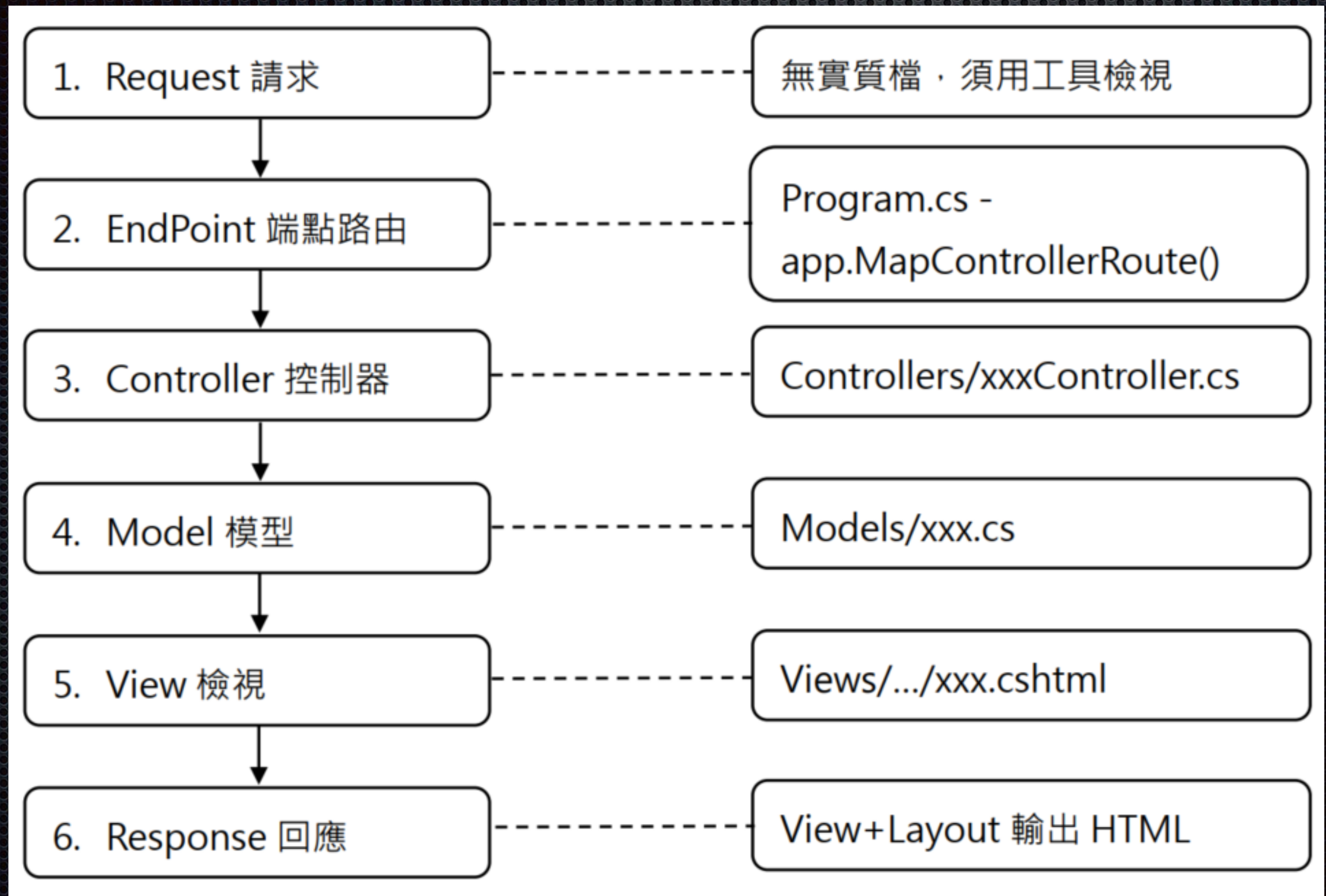
ASP.NET Core MVC專案結構



MVC專案執行畫面



ASP.NET MVC六大執行過程



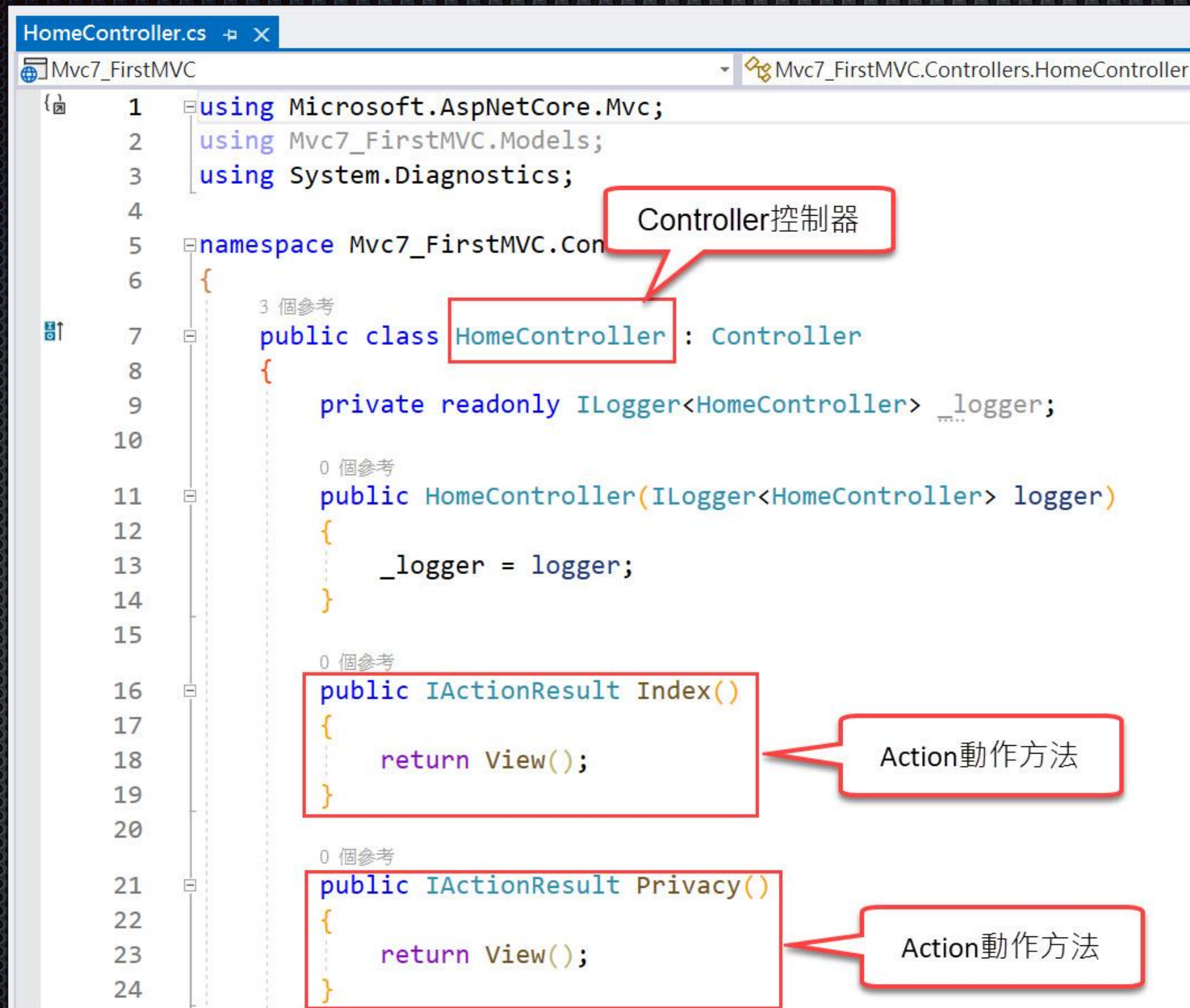
EndPoint端點路由

```
app.UseRouting();
```

```
...
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Controller 控制器



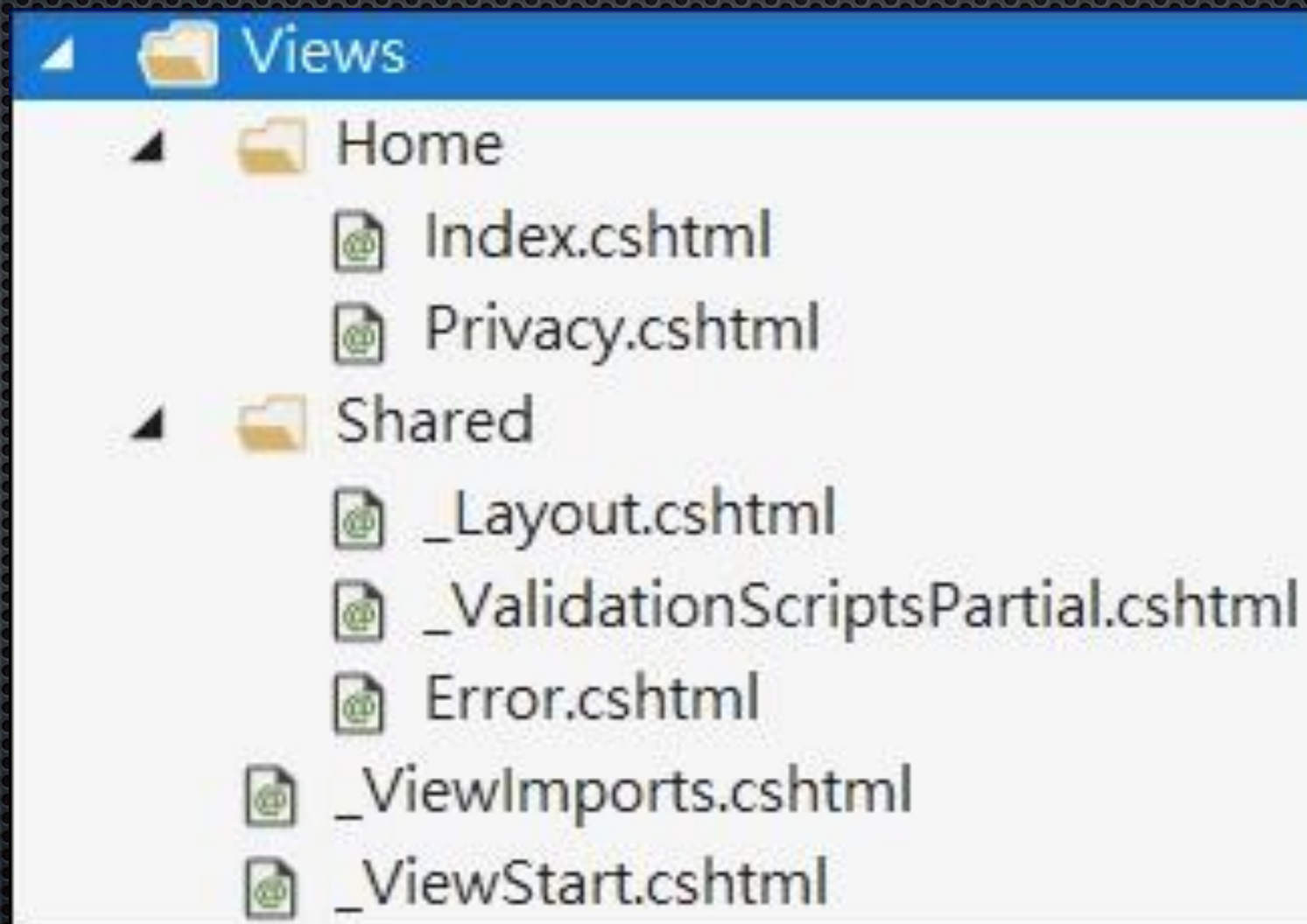
Model模型 (Product產品模型)

- Model模型是用來持有資料，模型名稱為單數

```
public class Product
{
    public int Id { get; set; }
    public string ProductName { get; set; }
    public int UnitPrice { get; set; }
}
```

Views檢視

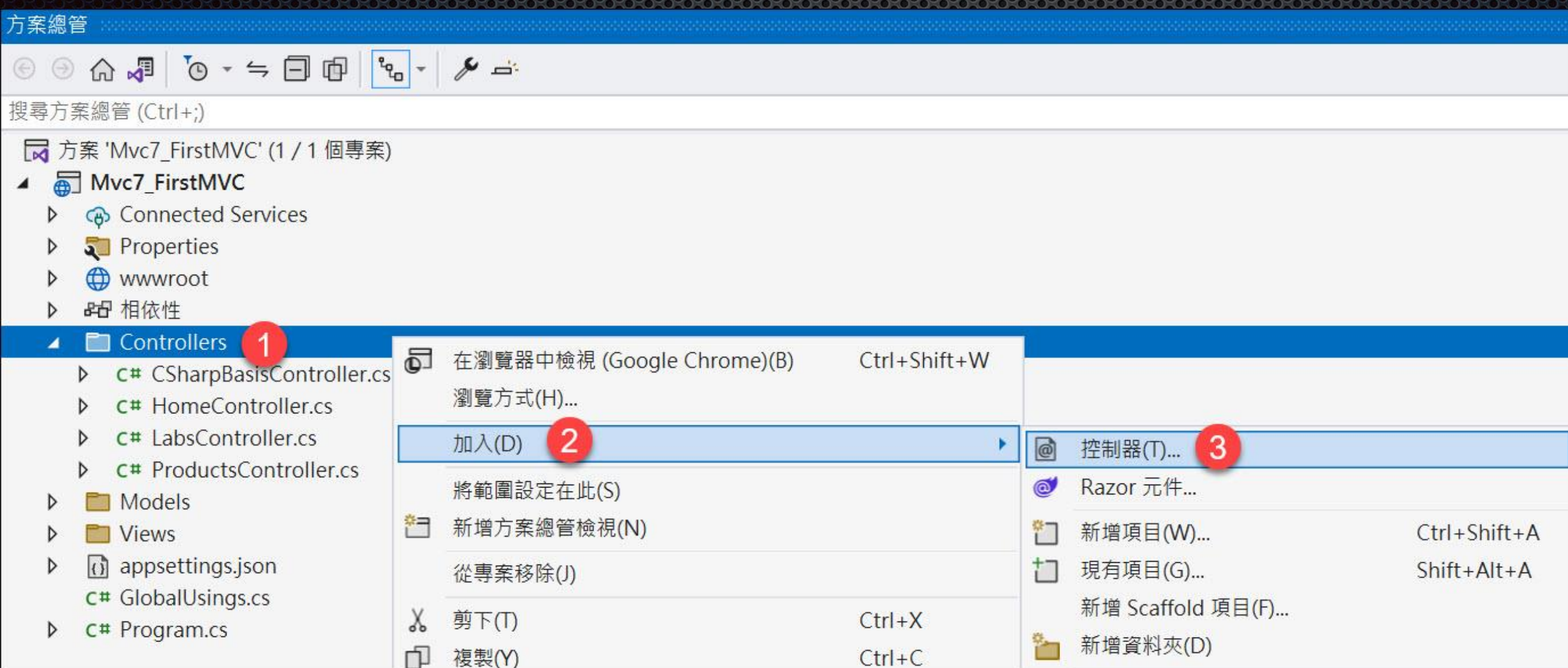
- View是用來設計網頁頁面
- .cshtml檔：Razor, html, css, js
- 通常一個Action對映一個View



掌握 Controller、Model 及 View 的建立技巧

2-5

加入Controller控制器



選擇「MVC 控制器 - 空白」樣板

新增 Scaffold 項目

▲ 已安裝

配置

▲ 通用

API

▲ MVC

控制器

檢視

Razor 元件

Razor 頁面

識別



MVC 控制器 - 空白

4



使用 Entity Framework 執行檢視的 MVC 控制器



執行讀取/寫入動作的 MVC 控制器

MVC 控制器 - 空白

由 Microsoft
v1.0.0.0

空白的 MVC 控制器。

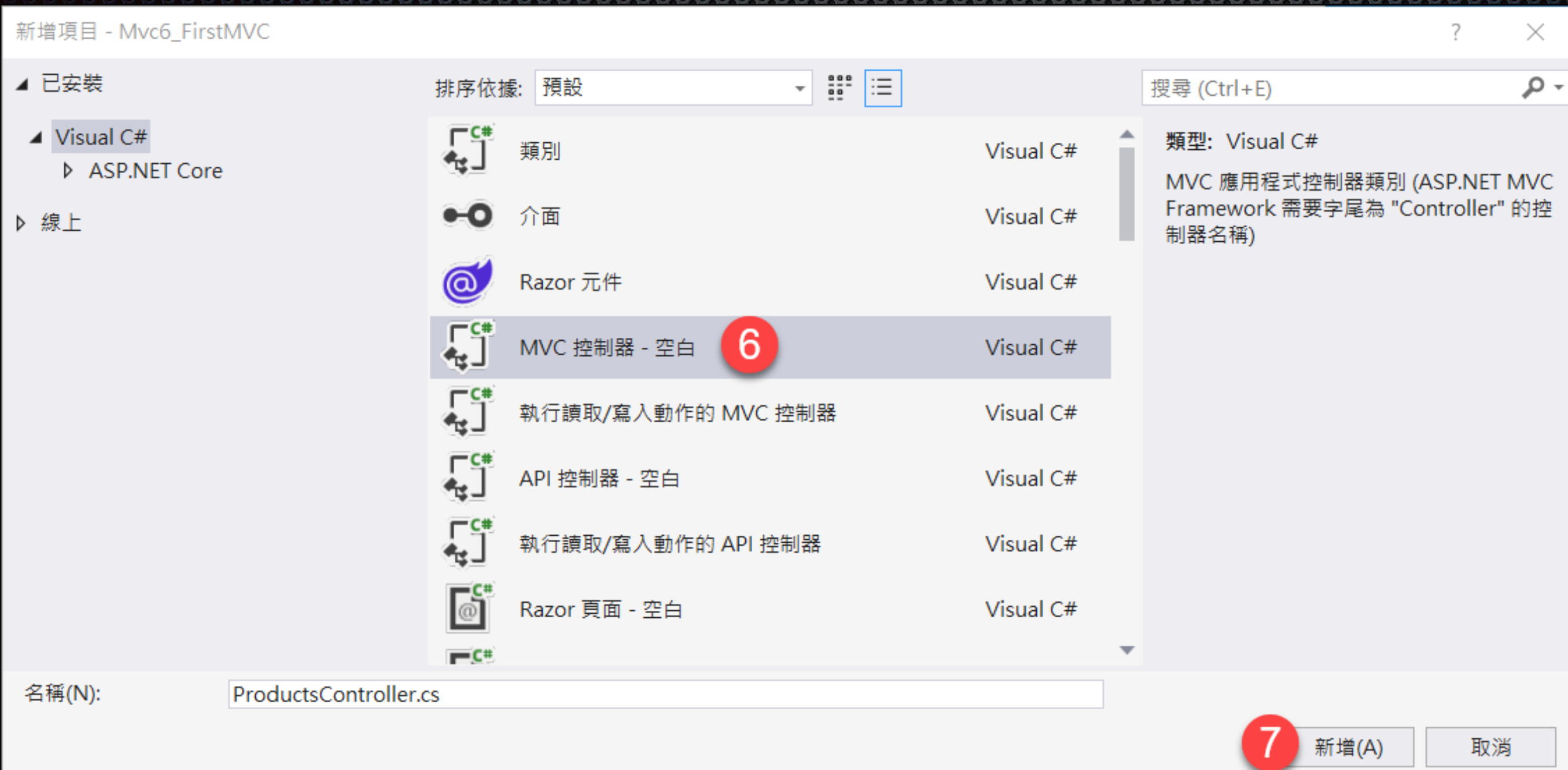
識別碼: MvcControllerEmptyScaffolder

5

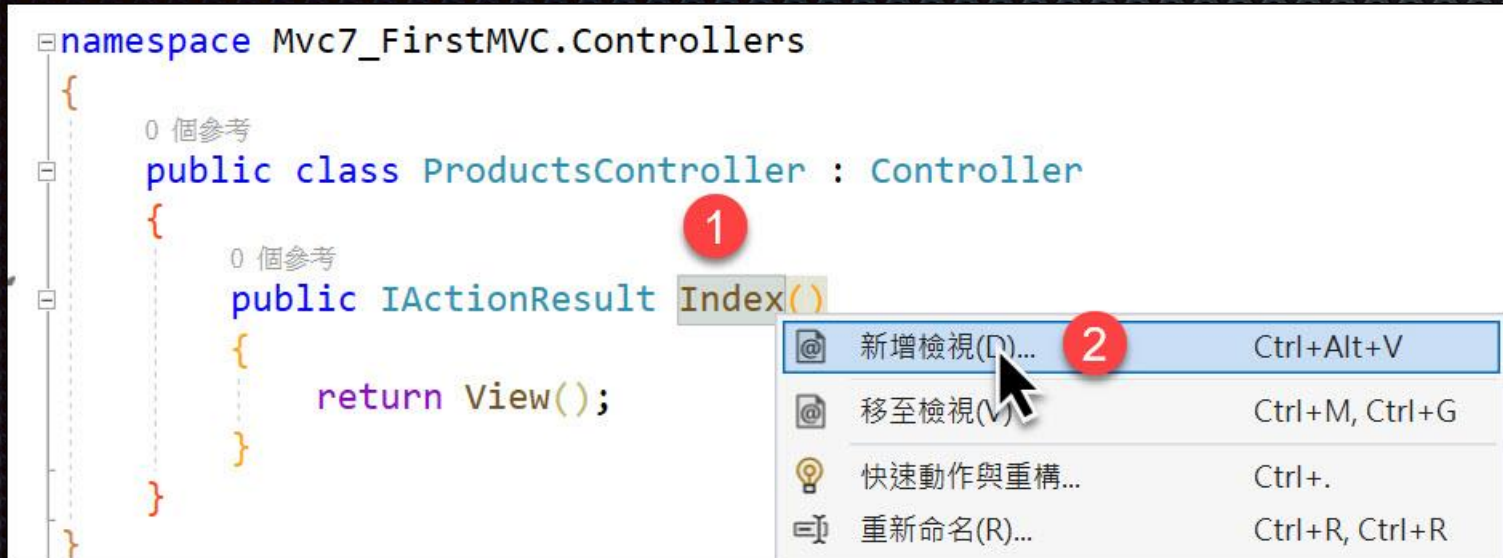
加入

取消

替控制器命名



新增View檢視



×

新增 Razor 檢視

檢視名稱(N)

Index 3

範本(T)

Empty (沒有模型) 4

模型類別(M)

選項

☐ 建立成局部檢視(C)

☒ 參考指令碼程式庫(R)

☒ 使用版面配置頁(U)

...

(若在 Razor _viewstart 檔案中設定，請保留空白)

5 新增

取消

Index.cshtml

```
@{  
    ViewData["Title"] = "Index";  
}
```

```
<h1>Index</h1>
```

Views/Products/Index.cshtml

三種 Index 頁面執行方式

- ① 第一種：在 Views/ Products /Index.cshtml 按滑鼠右鍵→【在瀏覽器中檢視】
- ② 第二種：修改Program.cs 中路由 controller 及 action 名稱，按 F5 執行
- ③ 第三種：在專案按滑鼠右鍵→【屬性】→【偵錯】→【一般】→【開啟debug啟動設定檔 UI】→【Mvc 7 _FirstMVC】→任選 http、https或 IIS Express三種執行方式之一→URL 輸入「Products/Index」→關閉對話視→再按F5執行

修改Program.cs 中路由 controller 及 action 名稱

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Products}/{action=Index}/{id?}");
```

Program.cs

在偵錯的【啟動瀏覽器】指定「Products/Index」

Mvc7_FirstMVC

搜尋屬性

- ▷ 全域 Using
- ▷ 應用程式
- ▷ 建置
- ▷ 套件
- ▷ Code Analysis
- ▷ Lint 分析
- ◀ 偵錯
 - 一般 1
- ▷ 資源
- ▷ TypeScript
- ▷ 快照偵錯工具

偵錯

一般

啟動設定檔的管理已移至專用對表，或透過標準工具列上的偵錯

2 開啟 debug 啟動設定檔 UI

資源

一般

組件資源的管理不再透過專案屬見，您可以透過下方連結存取。

[建立或開啟組件資源](#)

TypeScript

一般

ECMAScript 版本

產生之 JavaScript 所要使用的 I

ECMAScript 5

.tsx 檔的編譯模式

為 .tsx 檔指定 JSC 程式碼編譯

啟動設定檔

- http
- https 3
- IIS Express

啟用熱重新載入

☒ 將程式碼變更套用至執行中的應用程式。

啟用機器碼偵錯

☐ 同時為受控碼和機器碼啟用偵錯，也稱為混合模式偵錯

啟用 SQL Server 偵錯

☐ 啟用 SQL 指令碼偵錯與預存程序偵錯。

啟動瀏覽器

☒ 表示網頁瀏覽器在偵錯專案時應自動啟動。

指定 Controller/Action

URL

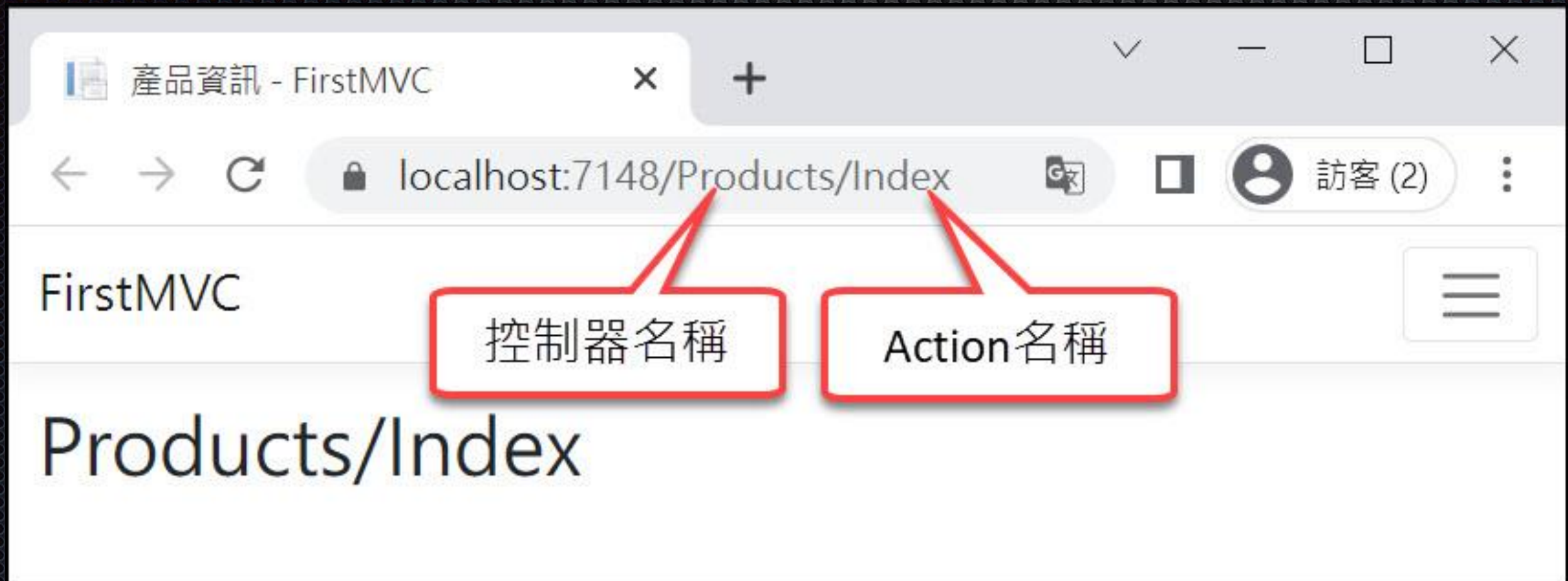
要瀏覽的網頁位址。

Products/Index 4

應用程式 URL

https://localhost:7148;http://localhost:5009

MVC網頁之URL代表意義



修改Port號碼

```
launchSettings.json  X
結構描述: https://json.schemastore.org/launchsettings.json
1  {
2    "profiles": {
3      "http": {
4        "commandName": "Project",
5        "launchBrowser": true,
6        "environmentVariables": {
7          "ASPNETCORE_ENVIRONMENT": "Development",
8        },
9        "dotnetRunMessages": true,
10       "applicationUrl": "http://localhost:5009"
11     },
12     "https": {
13       "commandName": "Project",
14       "launchBrowser": true,
15       "launchUrl": "Products/Index",
16       "environmentVariables": {
17         "ASPNETCORE_ENVIRONMENT": "Production",
18       },
19       "dotnetRunMessages": true,
20       "applicationUrl": "https://localhost:7148;http://localhost:5009"
21     },
22   },
23 }
```

Port號碼

Port號碼

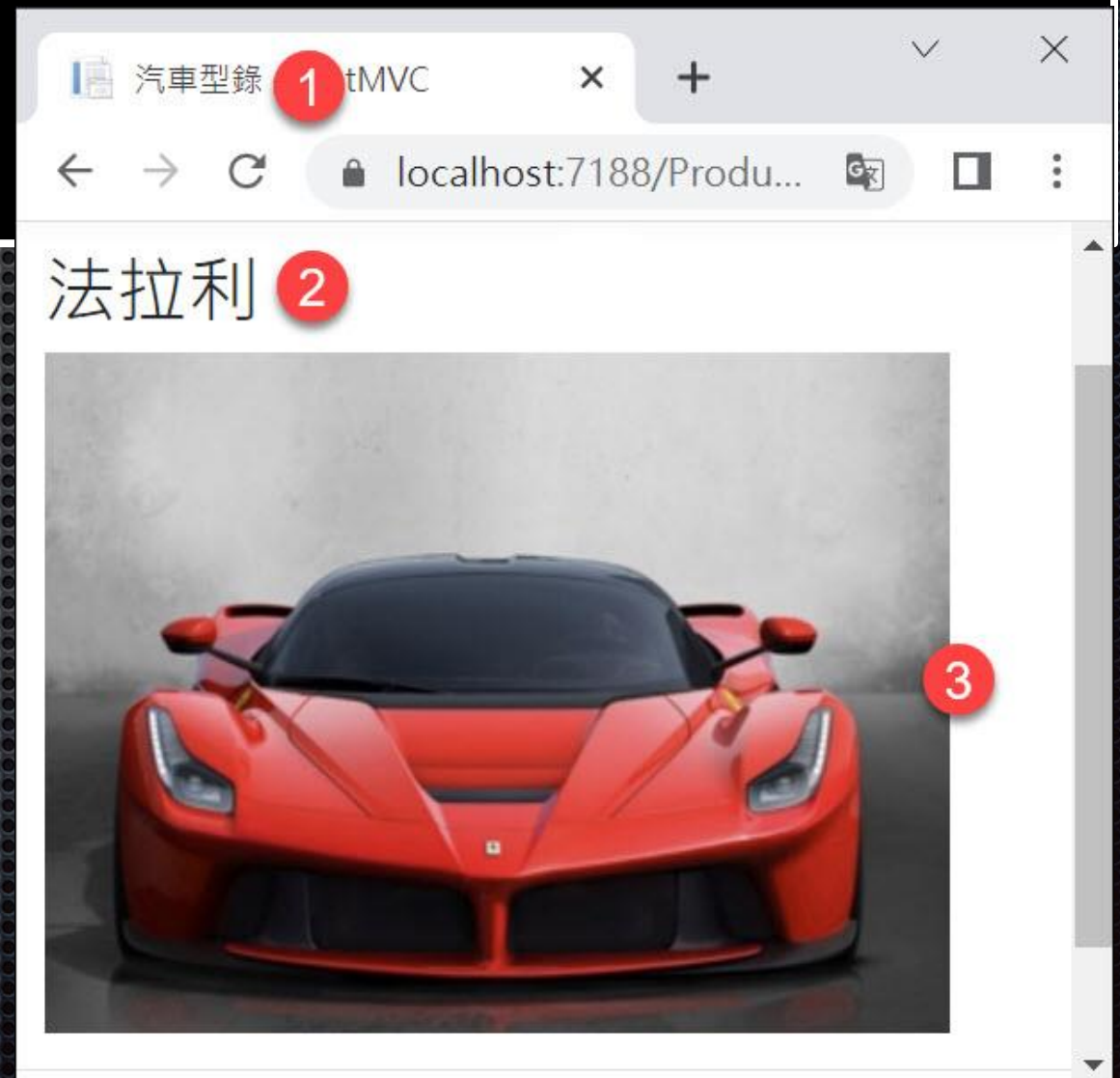
Port號碼

Properties/launchSettings.json

修改View檢視程式

```
@{  
    ViewData["Title"] = "汽車型錄";  
}  
  
<h1>法拉利</h1>  

```



建立Model模型

- Product 模型是用來持有產品資料
- 在 Models 資料夾按滑鼠右鍵→【加入】→【類別】→命名「Product.cs」→【新增】，並加入 Id、ProductName 和 UnitPrice 三個屬性

```
public class Product
{
    public int Id { get; set; }
    public string ProductName { get; set; }
    public int UnitPrice { get; set; }
}
```

Models/Product.cs

解析 ASP.NET Core MVC 專案資料夾功用

2-6

MVC 框架資料夾功用

本機電腦環境組態

相依性軟體

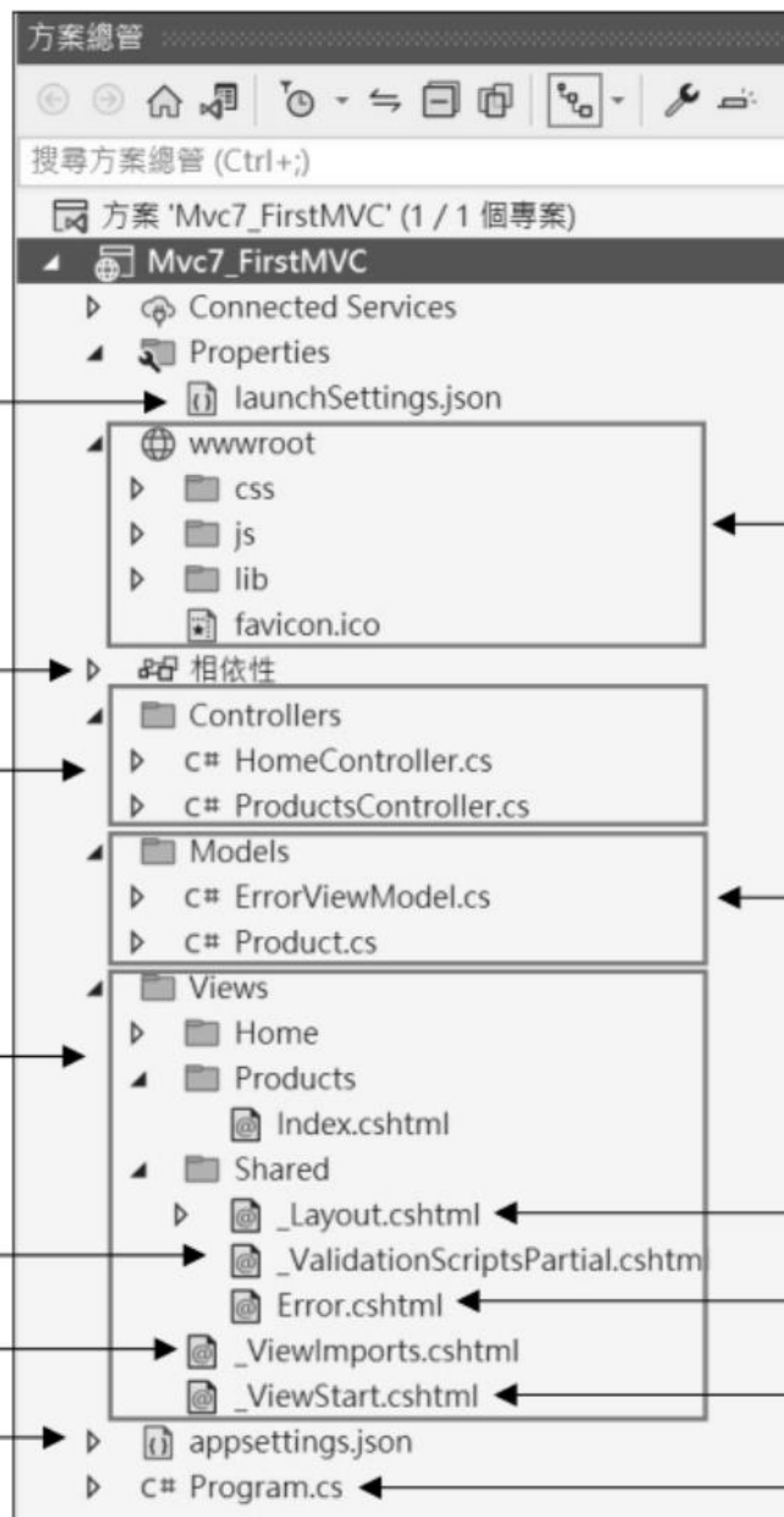
控制器

View 檢視

部分檢視

檢視輸入

App 組態



公開資源檔

Model 模型

佈局檔

錯誤 View 檢視

View 啟動執行

執行主程式

- **launchSettings.json**：本機開發電腦環境組態檔
- **wwwroot** 資料夾：公開的靜態資源檔目錄，如 css、js、lib 和 images
- **Controllers** 資料夾：Controller 控制器類別所在目錄
- **Models** 資料夾：Model 模型類別所在目錄
- **appsettings.json**：供應用程式使用的組態設定
- **Program.cs**：程式進入點，主要是創建 Host、環境及組態設定，負責 DI Container及Middleware元件設定

- **Views** 資料夾，View 檢視檔所在資料夾，其下有 Home 及 Shared 兩個子目錄，以及 `_ViewImports.cshtml` 和 `_ViewStart.cshtml` 檔
 - **Home** 目錄中有 Index、Privacy 二個 `.cshtml` 檢視檔
 - **Shared** 目錄中有 `_Layout.cshtml`、`_ValidationScriptsPartial.cshtml` 和 `Error.cshtml` 部分檢視檔
 - **`_ViewImports.cshtml`**：設定 Views 共享的指示詞，包括 `@using`、`@model`、`@inject`、`@inherits`、`@addTagHelper`、`@removeTagHelper` 和 `@tagHelperPrefix`
 - **`_ViewStart.cshtml`**：在每個 View 之前執行的程式碼

身分驗證的四種模式

2-7

身分驗證的四種模式

- 無驗證 (No Authentication)
- 個別使用者帳戶 (Individual User Accounts)
- Microsoft 身份識別平台 (Microsoft Identity platform)
- Windows 驗證 (Windows Authentication)

建立MVC專案時指定驗證

其他資訊

ASP.NET Core Web 應用程式 (Model-View-Controller)

C# Linux macOS Windows 雲端 服務 Web

架構(F) ⓘ

.NET 7.0 (標準字詞支援)

驗證類型(A) ⓘ

無

無

個別帳戶

Microsoft 識別平台

Windows

Linux

☐ 不要使用最上層陳述式(T) ⓘ

上一步(B) 建立(C)

在網頁會產生Register/Login

The screenshot shows a web browser window with the URL `localhost:7106/Identity/Account/Register`. The page title is "Register - Identity". The navigation bar includes links for "Identity", "Home", and "Privacy", and buttons for "Register" (annotated with a red circle 1) and "Login". The main content area is titled "Register" with the subtitle "Create a new account." and a link to "Use another service to register." Below this, there are three input fields: "Email" (containing "kevin@gmail.com"), "Password" (containing "....."), and "Confirm Password" (containing "....."). These fields are grouped by a red box and annotated with a red circle 2. Below the input fields is a blue "Register" button, annotated with a red circle 3. A message states: "There are no external authentication services configured. See this [article about setting up this ASP.NET application to support logging in via external services.](#)" The footer contains the text "© 2023 - Identity - [Privacy](#)".

Register - Identity

localhost:7106/Identity/Account/Register

Identity Home Privacy

1 Register Login

Register

Create a new account.

Use another service to register.

Email
kevin@gmail.com

Password
.....

Confirm Password
.....

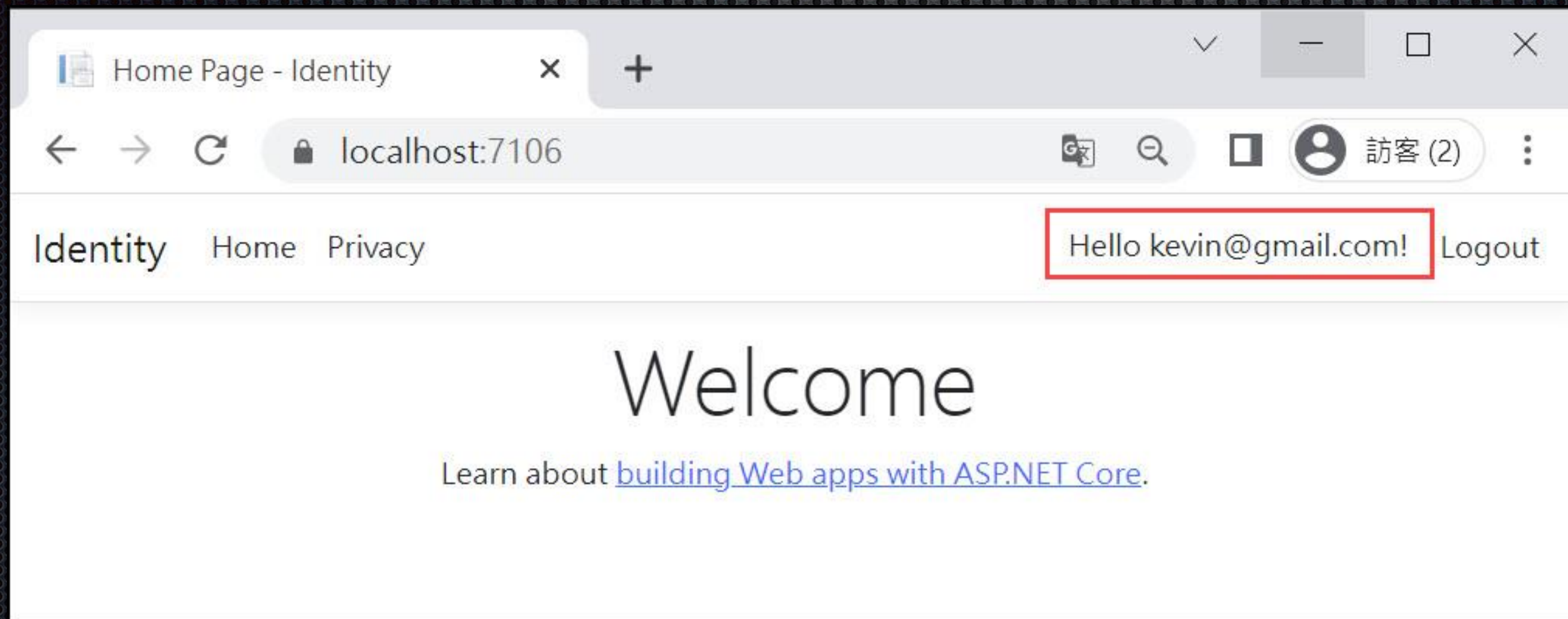
2

3 Register

There are no external authentication services configured. See this [article about setting up this ASP.NET application to support logging in via external services.](#)

© 2023 - Identity - [Privacy](#)

帳號登入成功狀態



用程式做身份驗證授權

```
using Microsoft.AspNetCore.Authorization;
```

```
[Authorize] ← 在控制器層級設定授權
```

```
public class HomeController : Controller  
{
```

```
...
```

```
[AllowAnonymous] ← 允許匿名存取
```

```
public IActionResult Index()  
{
```

```
    return View();  
}
```

```
public IActionResult Privacy()  
{
```

```
    if (User.Identity.Name != "Kevin@gmail.com")
```

利用 Identity.Name 判斷是否為特定使用者

```
    {  
        return Content($"{User.Identity.Name}無權存取此 Action 動作方法!");  
    }
```

```
    return View();  
}
```

授權予特定角色

```
[Authorize(Roles = "Admin, Supervisor")]
```

```
public IActionResult Contact()  
{
```

```
    return View();  
}
```

```
}
```

Identity身份驗證資料庫

SQL Server 物件總管

IdentityDB

- 資料表
 - 系統資料表
 - 外部資料表
 - dbo.__EFMigrationsHistory
 - dbo.AspNetRoleClaims
 - dbo.AspNetRoles
 - dbo.AspNetUserClaims
 - dbo.AspNetUserLogins
 - dbo.AspNetUserRoles
 - dbo.AspNetUsers
 - dbo.AspNetUserTokens
- 檢視

dbo.AspNetUsers [資料]

最大資料列(O): 1000

	Id	UserName	NormalizedUserN...	Email	NormalizedE...	EmailConfirm...
▶	4-bc203e92212b	john@gmail.com	JOHN@GMAIL.COM	john@gmail.co...	JOHN@GMAIL....	True
	7aa3dfb3-87bf-...	mary@gmail.com	MARY@GMAIL.COM	mary@gmail.c...	MARY@GMAIL...	True
	b4967460-f23b-...	Cindy@gmail.com	CINDY@GMAIL.CO...	Cindy@gmail.c...	CINDY@GMAI...	True
	e473fb0e-dce2-...	Kevin@gmail.com	KEVIN@GMAIL.COM	Kevin@gmail.c...	KEVIN@GMAIL...	True
*	NULL	NULL	NULL	NULL	NULL	NULL

用 LibMan 管理前端函式庫

2-8

LibMan(Library Manager)

- LibMan 負責前端函式庫的安裝、更新或移除管理
- 前端函式庫指的是 JavaScript 或 CSS Library，例如 jQuery、Chart.js 或 Bootstrap 函式庫
- LibMan 會從 CDN 下載，有 cdnjs、jsdelivr 和 unpkg 三個 CDN 來源，以及一個 filesystem，CDN 預設會使用 cdnjs
- LibMan 使用上又分兩種，一是經由 Visual Studio 的 GUI 介面，另一是透過 .NET Core CLI 命令

用Libman安裝Bootstrap

新增用戶端程式庫

提供者(P): unpkg 1

程式庫(L): bootstrap@5.2.3 2

☒ 包含所有程式庫檔案(A)

☐ 選擇特定檔案(S):

- ☒ 檔案:
 - ☒ dist
 - ☒ css
 - ☒ bootstrap-grid.css
 - ☒ bootstrap-grid.css.map
 - ☒ bootstrap-grid.min.css
 - ☒ bootstrap-grid.min.css.map

目標位置(T): wwwroot/lib/bootstrap/

3 安裝(I) 取消(C)

安裝後會產生libman.json檔

用LibMan解除安裝函式庫

libman.json

```
libman.json  X
結構描述: https://json.schemastore.org/libman.json
1  {
2    "version": "1.0",
3    "defaultProvider": "cdnjs",
4    "libraries": [
5      {
6        "library": "bootstrap@5.2.3",
7        "destination": "wwwroot/lib/bootstrap/"
8      },
9      {
10       "library": "jquery@3.6.3",
11       "destination": "wwwroot/lib/jquery/"
12     }
13   ]
14 }
```

解除安裝 bootstrap@5.2.3
檢查更新
排序屬性

IIS Express 及 SQL Server Express LocalDB 開發環境

2-9

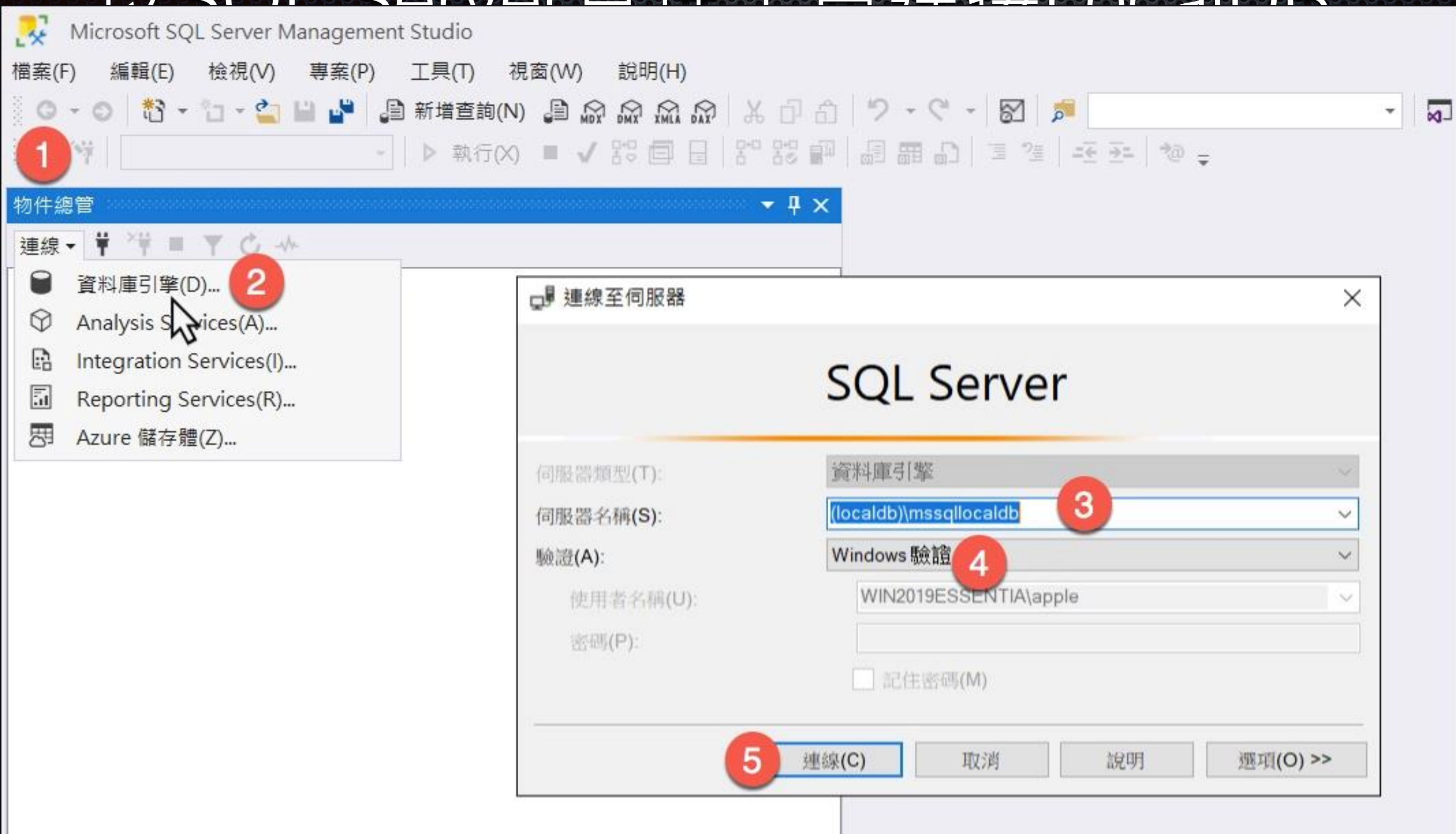
IIS Express 10

- 開發者電腦不需安裝IIS完整版
- 執行IIS Express大部分工作不需管理者權限
- 可與完整版IIS或其他Web伺服器並存安裝，且每個專案可選擇不同的Web伺服器。
- 多個使用者可以在相同的電腦上獨立操作
- IIS Express 可在 Windows 7 SP 1以後版本上執行

LocalDB

- LocalDB是SQL Server Express Database Engine的輕量級版本，專供開發人員使用
- Visual Studio安裝時會一併安裝LocalDB

以SQL Server管理工具連接localDB



SSMS下載網址：<https://bit.ly/3sXSmKc>

以Visual Studio開啟LocalDB

Visual Studio Interface showing the steps to open LocalDB data:

1. Select the database file (northwnd.mdf) in the Solution Explorer (方案總管).
2. Select the database (northwnd.mdf) in the Server Explorer (伺服器總管).
3. Right-click the selected table (Employees) and select "Show table data" (顯示資料表資料(S)).
4. The data is displayed in the Data Explorer (資料總管) under the table name (dbo.Employees [資料]).

EmployeeID	LastName	FirstName	Title	TitleOfCou...	Birth
1	Davolio	Nancy	Sales Repre...	Ms.	1948
2	Fuller	Andrew	Vice Presid...	Dr.	1952
3	Leverling	Janet	Sales Repre...	Ms.	1963
4	Peacock	Margaret	Sales Repre...	Mrs.	1937
5	Buchanan	Steven	Sales Mana...	Mr.	1955
6	Suyama	Michael	Sales Repre...	Mr.	1963
7	King	Robert	Sales Repre...	Mr.	1960
8	Callahan	Laura	Inside Sale...	Ms.	1958
9	Dodsworth	Anne	Sales Repre...	Ms.	1966
			NULL	NULL	NUL

在LocalDB建立Northwind資料庫

- 若LocalDB沒有Northwind資料庫的話，請至 <https://bit.ly/3pVCfNj> 全選並複製產生Northwind資料庫的SQL命令→在SSMS按下【新增查詢】→貼上SQL命令→按下【執行】按鈕，完成後即可看到Northwind資料庫

SQL Server 資料庫檔案路徑

- 系統資料庫: master、model、msdb、tempdb
 - C:\Users\<User>\AppData\Local\Microsoft\Microsoft SQL Server Local DB\Instances\mssqllocaldb
- 自行新增資料庫
 - C:\Users\<User>

IIS Server 及 Hosting Bundle 安裝

2-10

部署到IIS Server之Runtime需求

- 開發者電腦：於 Visual Studio Installer 中勾選安裝「開發時間 IIS 支援」(Development time IIS support)
- 獨立 IIS 伺服器或開發者電腦：安裝「ASP.NET Core 7.0 Runtime & Hosting Bundle for Windows」

開發時間 IIS 支援

安裝詳細資料

▼ ASP.NET 與網頁程式開發 *

▼ 已包含

- ✓ .NET Framework 4.7.2 開發工具
- ✓ ASP.NET 與網頁程式開發必要條件

▼ 選擇性

- ✓ .NET Framework 4.8 開發工具
- ✓ .NET Framework 專案與項目範本
- ✓ 網頁程式開發的雲端工具
- ✓ IntelliTrace
- ✓ .NET 分析工具
- ✓ Entity Framework 6 工具
- ✓ Live Unit Testing
- ✓ Live Share
- ✓ 使用 WSL 進行 .NET 偵錯
- ✓ IntelliCode
- ☐ Windows Communication Foundation
- ☒ 開發時間的 IIS 支援
- ☐ .NET Framework 4.6.2-4.7.1 開發工具
- ☐ 架構與分析工具
- ☐ Web 效能與負載測試工具
- ☐ 其他專案範本 (舊版)

總共所需空間 41 MB

在下載時安裝



修改(M)

ASP.NET Core 7 Runtime & Hosting Bundle for Windows

7.0.1 **Security patch** ⓘ
[Release notes](#) Latest release date December 13, 2022

Build apps - SDK ⓘ

SDK 7.0.101

OS	Installers	Binaries
Linux	Package manager instructions	Arm32 Arm32 Alpine Arm64 Arm64 Alpine x64 x64 Alpine
macOS	Arm64 x64	Arm64 x64
Windows	Arm64 x64 x86 winget instructions	Arm64 x64 x86
All	dotnet-install scripts	

Visual Studio support

Visual Studio 2022 (v17.4)

Visual Studio 2022 for Mac (v17.4)

Included in

Run apps - Runtime ⓘ

ASP.NET Core Runtime 7.0.1

The ASP.NET Core Runtime enables you to run existing web/server applications. On Windows, we recommend installing the Hosting Bundle, which includes the Runtime and IIS support.

IIS runtime support (ASP.NET Core Module v2)

17.0.22323.1

OS	Installers	Binaries
Linux	Package manager instructions	Arm32 Arm32 Alpine Arm64 Arm64 Alpine x64 x64 Alpine
macOS		Arm64 x64
Windows	Hosting Bundle x64 x86 winget instructions	Arm64 x64 x86

Hosting Bundle

部署 ASP.NET Core 應用程式 至 IIS 網站

2-11

Windows 的 IIS 及 ASP.NET Core 環境確認

- 需有 Windows Server，例如 Windows Server 2012、2016 或 2019，作為正式營運網頁伺服器
- 開發者 Windows 10 電腦亦可安裝 IIS 做同樣的事，只不過是開發者本人測試用，非正式營運網頁伺服器

Windows Server需安裝IIS伺服器





選取伺服器角色

目的地伺服器
Win2012R2

在您開始前

安裝類型

伺服器選取項目

伺服器角色

功能

確認

結果

選取一或多個要安裝在選取之伺服器上的角色。

角色

☐	網頁伺服器 (IIS) (8 / 43 已安裝)	▲
☐	網頁伺服器 (7 / 34 已安裝)	▶
☐	一般 HTTP 功能 (4 / 6 已安裝)	▶
☐	安全性 (1 / 9 已安裝)	▶
☐	狀況及診斷 (1 / 6 已安裝)	▶
☐	效能 (1 / 2 已安裝)	▶
☒	應用程式開發	▶
☐	.NET 擴充性 3.5	
☒	.NET 擴充性 4.5	
☐	ASP	
☐	ASP.NET 3.5	
☒	ASP.NET 4.5	
☐	CGI	
☒	ISAPI 篩選器	
☒	ISAPI 擴充程式	

描述

ASP.NET 提供伺服器端物件導向程式設計環境，可讓使用者利用 Managed 程式碼建立網站及 Web 應用程式。ASP.NET 4.5 不單只是新版本的 ASP，更以 .NET Framework 為基礎而完整重新建構來提供高生產力程式設計體驗，進而提供建立 Web 應用程式的健全基礎結構。

< 上一步(P)

下一步(N) >

安裝(I)

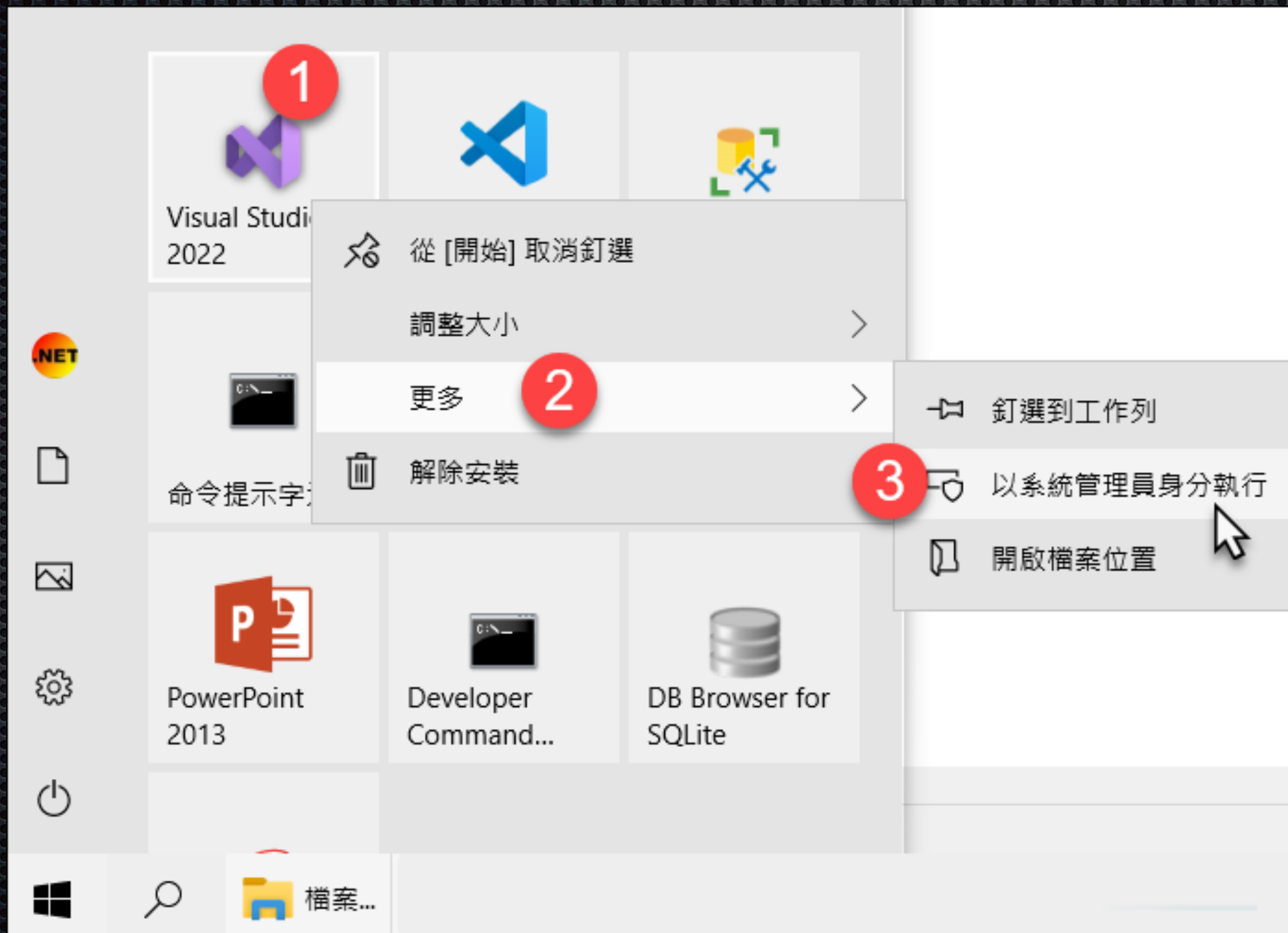
取消

Windows 10安裝IIS

- Windows 10本身IIS作測試，請搜尋並開啟【控制台】→【程式和功能】→【開啟或關閉Windows功能】→將Internet Information Services中【應用程式開發功能】的【ASP.NET 4.8】打勾→按【確定】進行安裝



以系統管理員權限開啟 Visual Studio



將MVC專案發佈至IIS

發佈

您今天在哪裡發佈？

目標



Azure

將您的應用程式發佈到 Microsoft 雲端



Docker 容器登錄

將您的應用程式發佈到所有可使用 Docker 映像的受支援容器登錄



資料夾

將您的應用程式發佈到本機資料夾或檔案共用



FTP/FTPS 伺服器

將您的應用程式發行到 FTP/FTPS 伺服器



網頁伺服器 (IIS)

使用 Web Deploy 或 Web Deploy 套件，將您的應用程式發佈到 IIS



匯入設定檔

匯入您的發行設定以部署您的應用程式

上一步(B)

下一步(N)

4

完成(F)

取消(C)

方案總管

搜尋方案總管 (Ctrl+;)

方案 'Mvc7_FirstMVC' (1 / 1 個專案)

1 Mvc7_FirstMVC

Connected Services

Properties

wwwroot

相依性

Controllers

Models

Views

appsettings.json

GlobalUsings.cs

libman.json

Program.cs

TextFile.txt

建置(U)

重建(E)

清除(N)

檢視(W)

分析與程式碼清除 (Z)

套件(P)

2 發佈(B)...

設定 Application Insights(C)...

概觀(V)

將範圍設定在此(S)

新增方案總管檢視(N)

檔案巢狀結構

編輯專案檔

加入(D)

管理 NuGet 套件(N)...

管理用戶端程式庫(M)...

管理使用者密碼(G)

移除未使用的參考...

同步命名空間(N)

設定為啟動專案(A)

偵錯(G)

剪下(T)

Ctrl+X

2 移除(V)

Del

設定發佈方法及IIS主機與虛擬目錄

發佈

發佈

連接
設定

IISProfile

發佈方法(M): Web Deploy

伺服器(E): Win10Station 7

網站名稱(T): Default Web Site/FirstMvc 8

使用者名稱(N):

密碼(W):

☐ 儲存密碼(A)

目的地 URL(L): 例如 *http://www.contoso.com*

驗證連線(V) ✓

< 上一個(R) 下一個(X) > 9 儲存 取消

執行發佈專案至 IIS 伺服器

The screenshot shows the 'Mvc7_FirstMVC: 發佈' (Mvc7_FirstMVC: Publish) window in Visual Studio. The window has a sidebar on the left with tabs: '概觀' (Overview), '已連線的服務' (Connected Services), and '發佈' (Publish), with '發佈' being the active tab. The main area shows the deployment configuration for 'IISProfile.pubxml' using 'Web Deploy'. A '發佈(U)' (Publish) button is visible, with a mouse cursor hovering over it. Below the button, a green message box states: '✓ 於 2022/12/28 的 下午 12:37 發佈成功。' (Successfully published on 2022/12/28 at 12:37 PM) with a link to '開啟網站' (Open Website). The '設定' (Settings) section lists various deployment options: '組態' (Configuration) set to 'Release', '部署模式' (Deployment Mode) set to 'Framework 相依性' (Framework Dependencies), and '目標執行階段' (Target Framework) set to '可攜' (Portable). A link '顯示所有設定' (Show All Settings) is provided. The '裝載' (Load) section shows the target server as 'Win10Station' and the application as 'Default Web Site/FirstMVC'. The '密碼' (Password) field is masked with '*****'. At the bottom, the '網站' (Website) URL is 'Http://localhost/FirstMVC'.

Mvc7_FirstMVC: 發佈

概觀

已連線的服務

發佈

IISProfile.pubxml

Web Deploy

發佈(U)

+ 新增 更多動作

✓ 於 2022/12/28 的 下午 12:37 發佈成功。
[開啟網站](#)

設定

組態 Release

部署模式 Framework 相依性

目標執行階段 可攜

[顯示所有設定](#)

裝載

伺服器 Win10Station

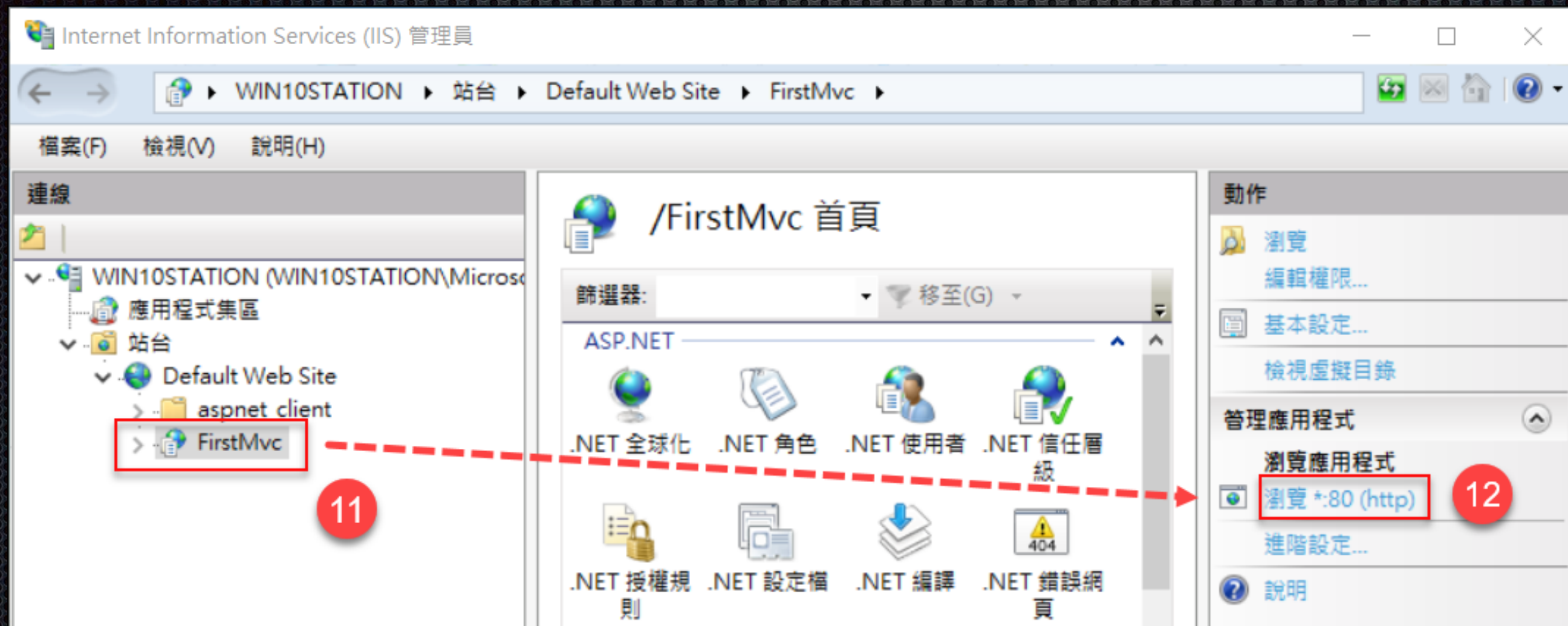
IIS 應用程式 Default Web Site/FirstMVC

使用者名稱

密碼 *****

網站: [Http://localhost/FirstMVC](http://localhost/FirstMVC)

在IIS管理員瀏覽MVC網站



建立 Model 時常用的 C#物件和集合初始設定式

2-12

物件初始設定式 (Object Initializers)

```
public class Friend
{
    public string Name { get; set; }
    public string Country { get; set; }
}
```

Model

```
public ActionResult CollectionInitializerswithNamedType()
```

```
{
```

集合初始設定式

```
//集合初始設定式
```

```
List<Friend> friends = new List<Friend>
```

```
{
```

```
    new Friend { Name = "Rose", Country = "USA" },
```

```
    ...
```

```
    new Friend { Name = "Max", Country = "Korea" }
```

```
};
```

```
return View(friends);
```

```
}
```

CollectionInitializerswithNamedType()

```
public ActionResult CollectionInitializerswithNamedType()
{
    //集合初始設定式
    List<Friend> friends = new List<Friend>
    {
        new Friend { Name = "Rose", Country = "USA" },

    };

    //以LINQ查詢泛型集合
    var friendsQuery = from f in friends
        where f.Country == "USA" || f.Country == "Korea"
        select f;

    var queryList = friendsQuery.ToList();

    var Query = from f in friends
        where f.Country == "USA" || f.Country == "Korea"
        orderby f.Country, f.Name
        select new { 姓名 = f.Name, 國家 = f.Country };

    return View(friendsQuery);
}
```

集合初始設定式

以LINQ查詢集合

The End

奚江華個人經歷簡介

- 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹、ASP.NET Core 3.x、ASP.NET Core 7 MVC跨平台範例實戰演練**
- 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- 曾數屆當選微軟MVP
- 國內各大上市公司及大專院校之培訓講師
- 曾發表文章刊於ITHOME電腦週刊

碼魔法FB & 網站

- CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw